

z/OS Job Control Language (JCL)

THE EXEC STATEMENT





EXEC STATEMENT



The EXEC or *execute* statement defines a job step.

It specifies the name of a program to execute, or the name of a JCL procedure to invoke in the step.

We will get to JCL procedures later. For now, we will specify a single program to execute in each job step.

JES requires each job to include at least one EXEC statement in addition to the JOB statement.

EXEC STATEMENT – POSITIONAL PARAMETER

```
//STEPNAME EXEC PGM=program_name
```

```
//STEPNAME EXEC PROC=procedure_name
```

```
//STEPNAME EXEC procedure_name
```

The EXEC statement takes a positional parameter that identifies the program or JCL procedure to be executed in the step. You can specify **PGM=** to identify a single program or **PROC=** to identify a procedure. Procedure is the default, so you can omit the **PROC=** keyword and code just the name of the procedure, if your intention is to run a procedure. Common practice is to omit the **PROC=** keyword.

This looks like a keyword parameter but it is really a positional parameter and must be the first parameter after the EXEC operation. It may refer either to a program or a procedure, so JES needs a way to know which one you intend. Hence, the **PGM=** and **PROC=** keywords.

PARM PARAMETER (KEYWORD)

```
//STEPNAME EXEC MYPROG
```

```
//      PARM=(150,ABC)
```

The PARM= parameter passes parameters to the program that is running in the current step. We'll see how to pass parameters to specific steps in JCL procedures later.

The executing program has to contain logic to parse and interpret the parameter values. The system does not parse program-specific parameters automatically.

Therefore, you will find different formats for parameters may be required, based on how each program was written. The example shows a common convention – a comma-separated list of values within parentheses – but this is not a hard-and-fast standard.

COND PARAMETER (KEYWORD)

```
//STEP4 EXEC MYPROG
```

```
//      PARM=(150,ABC) ,COND=(8,LT,STEP2)
```

COND is short for "condition." It refers to the *condition code* with which a previous job step completed. Condition code zero means successful completion.

There are no standards for condition code values, but by convention a code of zero indicates success, 12 indicates a fatal error, 8 indicates a serious error but the results of the step may still be valid, and 4 indicates a warning and the results of the step are probably valid.

The example specifies that STEP4 will be executed if the condition code from STEP2 was less than 8.

The COND parameter will make more sense when we get to the point that we're writing jobs that contain multiple steps. We will defer the details until then.

FREQUENTLY-USED EXEC STATEMENT PARAMETERS

The EXEC statement parameters described in this module are sufficient for you to complete the course, as well as for the majority of work you will do in real life.

A number of additional parameters exist for the EXEC statement. They are used to fine-tune the way JES and z/OS handle the step. They are out of scope for an introductory-level course on JCL.

Documentation on the positional and the most frequently-used keyword parameters on the EXEC statement is available at this URL:

```
https://www.ibm.com/docs/en/zos-basic-skills  
?topic=do-jcl-exec-statements-positional-frequently-used-parameters
```

SYSTEM OUTPUT FROM A JOB

Every job produces three standard data sets that go to the system spooler rather than to regular files. They are

- JESMSG LG
- JESJCL
- JESYSMSG

You control whether they are produced using the MSGLEVEL= parameter of the JOB statement. During the course, we'll specify MSGLEVEL=(1,1) so that the maximum system output is produced.

JESMSG LG is the JES Message Log. It contains all the system-generated messages pertaining to the job.

JESJCL contains the submitted JCL with symbolics resolved and JCL procedures expanded, so you can see the effective JCL that JES ran.

JESYSMSG contains messages pertaining to the execution of each job step, including what happened to data sets and whether any abends (abnormal ends) occurred.

LABS 13 AND 14

Refer to labs/labs.md in the course repo for instructions to complete Labs 13 and 14.

In these labs, you'll submit JCL and review the spooler output data sets from the job. You'll also clean up your "code" – that is, your Ansible playbooks – for submitting jobs.

WHAT YOU KNOW AND WHAT YOU CAN DO

- You understand the basics of the JOB and EXEC statements in JCL
- You can find the job output under SDSF and browse the various output data sets.
- You can write an Ansible playbook that captures one of these output data sets and copies its contents to a local file.

WHAT'S NEXT

- Sequential data sets under z/OS.
- The Data Definition (DD) JCL statement.
- The IEFBR14 utility.